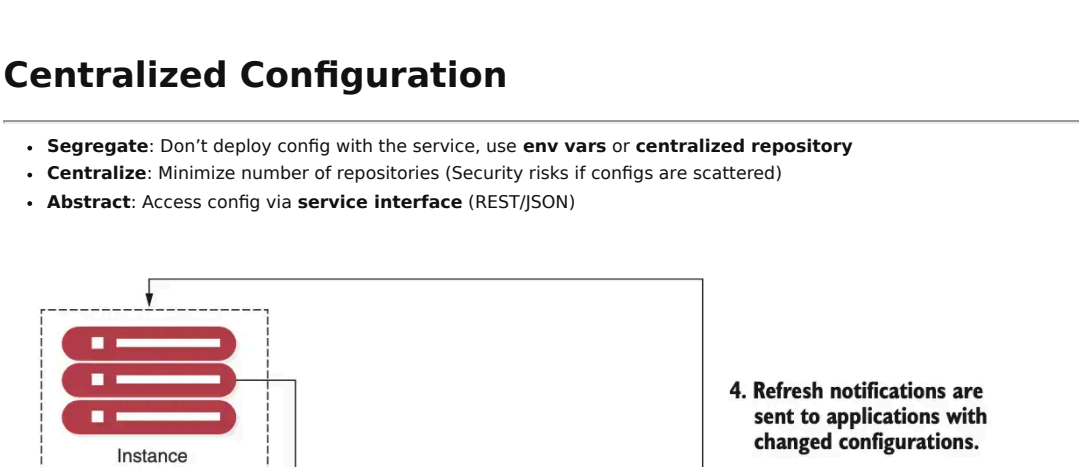
Centralized Configuration

- **Segregate**: Don’t deploy config with the service, use **env vars** or **centralized repository**
- **Centralize**: Minimize number of repositories (Security risks if configs are scattered)
- **Abstract**: Access config via **service interface** (REST/JSON)


Instance
Instance
Instance
Configuration management service
4. Refresh notifications are sent to applications with changed configurations.
2. Actual configuration resides in a repository.
Configuration service repository
Build/deployment pipeline
3. Changes from developers are pushed through the build and deployment pipeline to the configuration repository.
1. Microservice instances start up and obtain configuration information.
DevOps

1. Microservice starts → reads **config for its environment**
2. Config can be stored in:
   - Files (source control)
   - Relational DBs
   - Key-value stores
3. Config changes flow through **build & deployment pipeline** (GitOps)
4. Services **refresh config** when changes occur

Implementation Options

Spring Cloud Config Server

- General config solution with **multiple backends**
- Supported stores: filesystem, Git, Redis, Consul
- Well integrated with **Spring microservices**

Kubernetes ConfigMap/Secrets

- Stores **non-sensitive configuration** as key-value pairs
- **Decouples config from container images** for environment portability
- Can be consumed as **environment variables, command-line args, or mounted files**

Apache Zookeeper

- **Mature and battle-tested**
- Can manage key-value configs
- **Complex** → use if already part of architecture

Spring Cloud Configuration Server

When it comes to setting up a config server, there are a number of options to consider:

- **Selecting a storage type** for the configuration repository
- **Deciding on the initial client connection**, either to the config server or to the discovery server
- **Securing the configuration**, both against unauthorized access to the API and by avoiding storing sensitive information in plain text in the configuration repository

Storage type

Spring Cloud Config server supports the storing of configuration files in a number of different backends:

- Git repository
- Local filesystem
- JDBC database
- HashiCorp Vault

See the reference documentation for the full list.

Initial client connection

- **Config Server First:** The client first connects to the Config Server to retrieve its configuration and then registers with the Discovery Server. This approach allows storing the Discovery Server's configuration in the Config Server but creates a single point of failure since Spring Config Server is not distributed.

- **Discovery Server First:** The client first connects to the Discovery Server to find an available Config Server instance, reducing the risk of a single point of failure. This approach requires setting `spring.cloud.config.discovery.enabled=true` (default is false) but adds an extra network round trip during startup.

See the reference documentation for more details.

The Config Server API

The config server exposes a REST API that can be used by its clients to retrieve their configuration. We will use the following endpoints in the API:

- */{microservice}/{profile}*: Returns the configuration for the specified microservice and the specified Spring profile.
- */encrypt* and */decrypt*: Endpoints for encrypting and decrypting sensitive information. These must be locked down before being used in production.
- */actuator*: The standard actuator endpoint exposed by all microservices. These should be used with care and locked down before being used in production.

Setting up Spring Cloud Config Server

Maven dependencies

```
<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2025.1.0</spring-cloud.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config-server</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Configuration

```
server:
  port: 8888

spring:
  application:
    name: config-server
  cloud:
    config:
      server:
        git:
          uri: https://github.com/nbicocchi/learn-microservices-config
          skipSslValidation: true
          timeout: 4

encrypt:
  key: ${CONFIG_SERVER_ENCRYPT_KEY}

eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
    initialInstanceInfoReplicationIntervalSeconds: 5
    registryFetchIntervalSeconds: 5
  instance:
    leaseRenewalIntervalInSeconds: 5
    leaseExpirationDurationInSeconds: 5

---
spring.config.activate.on-profile: docker

server:
  port: 8080

eureka:
  client:
    serviceUrl:
      defaultZone: http://eureka:8761/eureka/
```

Server code

```
@SpringBootApplication
@EnableConfigServer
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Docker configuration

```
  config:
    build: config-service
    ports:
      - "8888:8888"
    environment:
      - SPRING_PROFILES_ACTIVE=${SPRING_PROFILES_ACTIVE}
      - ENCRYPT_KEY=${CONFIG_SERVER_ENCRYPT_KEY}
    healthcheck:
      test: [ "CMD-SHELL", "curl -f http://localhost:8888/actuator/health" ]
      interval: 10s
      timeout: 5s
      retries: 5
    depends_on:
      eureka:
        condition: service_healthy
```

The values of the preceding environment variables, marked in the Docker Compose file with ${...}, are fetched from the `.env` file:

```
CONFIG_SERVER_ENCRYPT_KEY=ninna-nanna-ninna-0h
SPRING_PROFILES_ACTIVE=docker
CONFIG_SERVER_HOST=config
CONFIG_SERVER_PORT=8888
```

These environmental variables can be injected in IntelliJ Configurations using third party plugins such as EnvFile.

Config Repository

After moving the configuration files from each client’s source code to the configuration repository, we will have some common configuration in many of the configuration files:

- the common parts have to be placed in a common configuration file *application.yml*
- this file is shared by all clients

```
config-repo/
├── application.yml
├── datetime-composite-service.yml
├── datetime-service.ym
```

The most of these files are simple and similar to each because all common parts have been included in *application.yml*.

```
eureka:
  client:
    serviceUrl:
      defaultZone: http://localhost:8761/eureka/
    initialInstanceInfoReplicationIntervalSeconds: 5
    registryFetchIntervalSeconds: 5
  instance:
    leaseRenewalIntervalInSeconds: 5
    leaseExpirationDurationInSeconds: 5

management:
  endpoints:
    web:
      exposure:
        include: health,info,env,refresh

---
spring.config.activate.on-profile: docker
eureka:
  client:
    serviceUrl:
      defaultZone: http://eureka:8761/eureka/
```

Trying out Spring Cloud Config Server

```
curl http://localhost:8888/datetime-service/docker | jq
```

```
{
  "name": "datetime-service",
  "profiles": [
    "docker"
  ]
}
```

The property sources are returned in priority order; if a property is specified in multiple property sources, the first property in the response takes precedence.

Securing the configuration

To protect configurations, Spring Cloud Config Server, **secures sensitive properties using encryption mechanisms (symmetric or asymmetric).**

```
curl http://localhost:8888/encrypt -d my-super-secure-password
```

```
curl http://localhost:8888/decrypt -d
4c9b0621e3b70d23e08b2868c07ca584bcaddbac798f1f9f2dbbc0262f3f2c0400c9a3d7fcf3481688256b252b5527bf
```

When the config server detects values in the format '{cipher}...', it tries to decrypt them using its encryption key before sending them to a client.

Setting up Spring Cloud Config Clients

Maven dependencies

```
<properties>
    <java.version>21</java.version>
    <spring-cloud.version>2025.1.0</spring-cloud.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.cloud</groupId>
        <artifactId>spring-cloud-config</artifactId>
    </dependency>
</dependencies>

<dependencyManagement>
    <dependencies>
        <dependency>
            <groupId>org.springframework.cloud</groupId>
            <artifactId>spring-cloud-dependencies</artifactId>
            <version>${spring-cloud.version}</version>
            <type>pom</type>
            <scope>import</scope>
        </dependency>
    </dependencies>
</dependencyManagement>
```

Updating the configuration

To update the configuration across all services, we must call the `/actuator/refresh` endpoint on each service. This can be achieved by using a simple shell script that iterates over the list of services and triggers the refresh.

```
#!/bin/bash

# List of service URLs (replace with actual service URLs)
SERVICES=("http://service1:8080" "http://service2:8080" "http://service3:8080")

# Loop through each service and trigger the /actuator/refresh endpoint
for SERVICE in "${SERVICES[@]}"
do
  echo "Refreshing config for $SERVICE"
  if curl -X POST "$SERVICE/actuator/refresh" -s -o /dev/null
    echo "✅ Config refreshed for $SERVICE"
  else
    echo "❌ Failed to refresh config for $SERVICE"
  fi
done
```

However, if we have replicated services or services are hidden behind a gateway, it becomes more complex, as the configuration must be refreshed on all instances of each service. In this case, we can implement a cross-cutting concern directly into the API gateway. By leveraging the Discovery Client and RestClient, the gateway can dynamically discover all replicas of a service and send a `/actuator/refresh` request to each instance. This approach ensures that configuration updates are propagated consistently across all replicas without the need for manually handling each service instance.

```
@Service
public class ServiceRefresher {
    private final ReactiveDiscoveryClient discoveryClient;
    private final WebClient webClient;

    public ServiceRefresher(ReactiveDiscoveryClient discoveryClient, WebClient.Builder webClientBuilder) {
        this.discoveryClient = discoveryClient;
        this.webClient = webClientBuilder.build();
    }

    public Mono<Void> refreshAllServices() {
        return discoveryClient.getServices() // Fetch all registered services reactively
                .flatMap(discoveryClient::getInstances) // Get instances for each service
                .flatMap(this::refreshInstance) // Call /actuator/refresh for each instance
                .then(); // Return Mono<Void>
    }

    private Mono<Void> refreshInstance(ServiceInstance instance) {
        String url = instance.getUri().toString() + "/actuator/refresh";
        System.out.println("Refreshing: " + url);

        return webClient.post()
                .uri(url)
                .retrieve()
                .bodyToMono(Void.class)
                .doOnSuccess(v -> System.out.println("✅ Refreshed: " + url))
                .doOnError(e -> System.err.println("❌ Failed to refresh " + url + " - " + e.getMessage()))
                .onErrorResume(e -> Mono.empty()); // Avoid breaking the chain if one request fails
    }
}
```

Resources

- Spring Microservices in Action (Chapter 5)
- Microservices with Spring Boot 3 and Spring Cloud (Chapter 12)